

Recover Pe.

- **Part 1: Tech Stack & System Architecture Overview** (Firebase + Supabase hybrid, Next.js, Tailwind).
- **Part 2: UI/UX & Design System** (Strict Black, White, & Grey UI, incorporating the Recoverpe Checkmark logo).
- **Part 3: Authentication Engine (Firebase)** (One-time Mobile OTP linked to Email).
- **Part 4: Workspace Architecture** (B2B Mode for MSMEs vs. Personal Mode for Friends/Family).
- **Part 5: The Relational Database Schema (Supabase)** (Ledgers, Invoices, and User relations).
- **Part 6: The Ingestion Engine** (CSV Bulk Uploads vs. Single manual entries).
- **Part 7: Zero-KYC Payment Routing** (Direct UPI Deep Link generation).
- **Part 8: The Omnichannel Escalation & Communication Engine**
- **Part 9: VAPI AI Voice Caller & Wallet System** (Triggering AI calls and deducting from the user's prepaid recharge balance).
- **Part 10: Automation & Cron Job Scheduler** (The custom due-date timelines).
- **Part 11: The Edge Case Manager** (Partial payments, offline cash, paused alerts).
- **Part 12: User Dashboard & Firebase Analytics** (Visualizing recovered money).
- **Part 13: File Storage (Firebase) & PDF Generation** (Hosting CSVs and creating invoices).
- **Part 14: Monetization (Razorpay)** (Freemium tier, Premium tier, and VAPI Wallet recharges under Dahikar Technologies Pvt Ltd).
- **Part 15: Super Admin Panel** (Our master view to track system health and user growth).

Part 1: Tech Stack & System Architecture Overview

(Copy everything below this line and paste it into a new Cursor chat to initialize the project)

System Prompt: You are an expert Founding CTO and Senior Full-Stack Next.js Developer. We are building "Recoverpe", a SaaS platform by Dahikar Technologies Private Limited. The platform automates accounts receivable and personal lending reminders via WhatsApp and AI Voice Calling.

Your objective for this prompt is to initialize the project, define the core hybrid tech stack, establish the folder structure, and document the global architectural rules. Do not write feature code yet; establish the environment.

1. The Hybrid Tech Stack

- **Framework:** Next.js 14+ (App Router).
- **Styling:** Tailwind CSS (Strictly using a minimalist Bright Mode: Black, White, and slight Grey highlights).
- **Authentication, Storage & Analytics:** Firebase. We will use Firebase for a one-time Mobile OTP verification that links permanently to the user's Email login. Firebase will also handle file storage and user analytics.
- **Primary Database:** Supabase (PostgreSQL). We require a strict relational database for complex multi-tenant SaaS logic (Ledgers, Invoices, Workspaces, Credit Wallets).
- **External APIs (To be integrated in later modules):**
 - Razorpay API (for SaaS Subscriptions and Wallet Recharges).
 - Official Meta WhatsApp Cloud API.
 - VAPI (AI Voice Calling).

2. Core Architecture Rules

- **Mobile-First Dashboard:** The UI must prioritize mobile responsiveness for business owners on the go, utilizing the Black/White/Grey aesthetic.
- **Workspace Separation:** The architecture must account for two distinct modes: "Business/MSME" (formal PDF invoices) and "Personal" (informal lending to friends/family without PDFs).
- **Server-Side Rendering (SSR):** Prioritize Next.js Server Components for data fetching from Supabase to ensure speed.
- **API Route Security:** All external API communications (Meta, Razorpay, VAPI) must occur securely via Next.js Route Handlers (`app/api/...`).
- **Type Safety:** The entire project must use strict TypeScript.

3. Initial Folder Structure Establish the following directory structure within the `src/` folder:

- `/app/api` - Next.js Route Handlers.
- `/app/(dashboard)` - The protected main user area.
- `/app/(auth)` - Firebase Login and OTP verification pages.
- `/components/ui` - Reusable Tailwind B&W components.

- `/lib` - Utility configs (`firebase.ts`, `supabase.ts`, etc.).
- `/types` - TypeScript interfaces.

Action Required from Cursor: Acknowledge these architectural rules. Then, generate the initial `package.json` file including the necessary dependencies (`next`, `react`, `tailwindcss`, `firebase`, `@supabase/supabase-js`) and provide the exact terminal commands required to initialize this Next.js project with these tools.

Part 2: UI/UX & Design System

1. Core Expectation & Vibe

The platform must feel like a high-end financial institution combined with the simplicity of a modern messaging app. Because we are dealing with money, debt, and ledgers, the design must communicate absolute trust, security, and clarity.

2. Theming & Color Palette

- **Strictly Bright Mode:** There will be no dark mode toggle. Financial ledgers and invoices are traditionally read on white backgrounds.
- **Primary Colors:**
 - **Pure White** (`#FFFFFF`): The background of the entire app.
 - **Pitch Black** (`#0A0A0A`): Used for all primary text, main action buttons, and logo elements to match the bold Recoverpe checkmark.
- **Secondary Colors (The Greys):**
 - **Light Grey** (`#F3F4F6`): Used for hover states, table headers, and dividing lines.
 - **Medium/Dark Grey** (`#4B5563`): Used for secondary text (like dates or timestamps) so it doesn't distract from the main numbers.
- **Status Highlight Colors (Used sparingly):**
 - **Success Green** (`#10B981`): Strictly reserved for badges that say "Paid", "Recovered", or the checkmark icon when a transaction is successful.
 - **Error Red** (`#EF4444`): Strictly reserved for "Overdue", "Failed", or destructive actions like "Delete Ledger".

3. Component Features & Rules

- **Flat UI Components:** We will avoid heavy drop-shadows or 3D effects. Cards, tables, and containers will use 1px solid light grey borders. This keeps the interface looking "crisp" and fast.
- **Buttons:**
 - **Primary Action Button:** Solid Black background with White text (e.g., "Send Reminder", "Save Invoice").
 - **Secondary Action Button:** White background with a 1px Black border and Black text (e.g., "Cancel", "Edit").

- **Input Fields:** Clean white boxes with grey borders. When a user clicks to type, the border turns solid black. No colored glows.
- **Typography:** We will use a highly legible, modern sans-serif font (like Inter or Roboto). Numbers (amounts, phone numbers) must use a tabular figure setting so that digits align perfectly in ledger columns.

4. Layout & Responsiveness

- **Mobile-First Construction:** 90% of Indian MSME owners will check this on their mobile browsers. Buttons must be thumb-sized. Tables must be horizontally scrollable on small screens without breaking the page layout.
- **Navigation:** A clean bottom navigation bar for mobile (Home, Ledgers, Wallet, Settings) and a collapsible side menu for desktop.

Part 3: Authentication Engine & Security

1. The Core Strategy: Email Primary + One-Time Mobile Verification

We are using Firebase for Authentication because of its highly reliable Phone OTP delivery and seamless Google integration. However, since B2B users often change phones or have multiple devices, we will not force them to use OTP *every time* they log in.

- **Primary Identity:** Email & Password (or 1-Click Google Sign-In).
- **Trust Verification:** A one-time Mobile OTP gate during the initial signup.
- This ensures every account is tied to a real, verified human (preventing spam accounts from abusing our WhatsApp/VAPI limits), while keeping daily logins frictionless.

2. The Step-by-Step User Flows

A. The Registration Flow (New Users)

1. **Step 1 (The Hook):** User clicks "Get Started" and enters Email/Password or clicks "Sign up with Google". Firebase creates the initial user object.
2. **Step 2 (The Gate):** Before they can see the dashboard, the app immediately routes them to a "Verify Mobile" screen.
3. **Step 3 (The OTP):** User enters their 10-digit Indian mobile number (+91). Firebase sends a 6-digit OTP via SMS (using an invisible reCAPTCHA to prevent bot abuse).
4. **Step 4 (The Link):** Upon entering the correct OTP, the mobile number is permanently linked to their Firebase Email account.

B. The Login Flow (Returning Users)

- Frictionless. The user simply logs in via Email/Password or Google.
- *No OTP is required* unless they are logging in from a highly suspicious IP or requesting a password reset.

3. Firebase to Supabase Data Synchronization (Critical Infrastructure)

Because Firebase handles the login, but Supabase holds all our actual business data (invoices, ledgers, wallets), the two systems must talk to each other instantly.

- **The Sync Logic:** The moment a user successfully completes the Mobile OTP verification in Firebase, our Next.js backend will silently fire an API call to Supabase.
- **The Action:** It creates a new row in our Supabase `Users` table with the following data:
 - `firebase_uid` (The unique ID from Firebase).
 - `email`
 - `verified_phone_number`
 - `account_status` (Active)
- **Rule:** Every subsequent read/write to the Supabase database will check the user's `firebase_uid` to ensure they only see their own business data.

4. Security & Spam Prevention Rules

- **Country Code Restriction:** To control initial API costs and comply with Meta's pricing tiers, the mobile input field will strictly lock the country code to India (+91) for now.
- **Rate Limiting:** Firebase App Check and invisible reCAPTCHA must be enabled to prevent competitors or bots from spamming our OTP endpoint and draining our SMS budget.
- **Phone Number Uniqueness:** A single phone number can only be linked to *one* Recoverpe account. If a user tries to verify a number that already exists in our system, they will be prompted to log into that existing account instead.

Part 4: Workspace Architecture & Profile Management

1. The Core Philosophy: "Always-On" Dual Mode

We will not force the user to choose a path during onboarding. The moment a user completes their OTP verification, their dashboard is fully unlocked with two permanent, side-by-side environments: **Business** and **Personal**.

There is no "logging in and out" or complex workspace creation just to lend ₹2,000 to a friend. Both modes are instantly accessible, but the features within them behave entirely differently.

2. The Personal Ledger (Instantly Active)

- **Purpose:** For informal lending, splitting bills, and tracking money owed by friends or family.
- **The Experience:** It is active by default. The user can immediately add a contact and an amount.
- **Feature Constraints:**
 - No business branding, no PDF generation, and no tax fields.
 - WhatsApp reminders are drafted in a highly conversational, friendly tone (e.g., *"Hey! Just a quick reminder about the ₹5000. Let me know when you can send it over. Here is my UPI link..."*).
 - VAPI AI voice calling is disabled for this mode (we don't want AI debt collectors calling someone's cousin).

3. The Business Profiles & Dynamic GST Logic

- **Purpose:** Formal accounts receivable, B2B ledgers, and official business transactions.
- **Creating a Business:** A user can click "Add Business Profile". They input their Business Name, Logo, and explicitly, their **GSTIN (GST Number)**.
- **The Dynamic Invoice Engine:**
 - **Scenario A (GSTIN Provided):** If the user inputs a valid GST number, the system automatically shifts to compliance mode. Every PDF generated is titled **"Tax Invoice"**, calculates CGST/SGST/IGST automatically based on the client's state code, and strictly formats to Indian government invoicing standards.
 - **Scenario B (No GSTIN):** If the business is unregistered (Turnover < ₹40L), the system generates a standard **"Bill of Supply"** or basic "Invoice" without tax breakdowns, keeping it legally compliant for small vendors.
- **Multi-Business Capability:** A single user can add multiple distinct businesses (e.g., "Rahul Traders" and "Rahul Electronics") and toggle between them.
- **WhatsApp Tone:** Reminders are strictly professional, formatted like official company notices, and include the generated PDF link alongside the payment link.

4. The UI/UX Switching Experience

- **The Toggle:** At the top center of the mobile screen (or top left on desktop), there is a sleek, persistent toggle switch: [**Personal**] | [**Business**].
- **Visual Distinction:** While the whole app retains the strict Black & White theme, switching between the two modes should have a subtle visual cue—perhaps the "Personal" dashboard feels slightly softer with rounded cards, while the "Business" dashboard is structured with rigid, analytical tables.

5. Database Schema Logic (For the Developer)

To support this "Always-On" structure, the database avoids complex "bridge" tables for solo users and keeps data perfectly segregated:

- **Users Table:** Stores the core identity (Firebase UID, Email, Phone).

- **Businesses Table:** Stores business entities (ID, User ID, Name, GSTIN, Logo).
- **Ledgers Table:** Every transaction must have a `type` column (`personal` or `business`). If it is a business transaction, it must also include the `business_id`. This guarantees personal transactions never accidentally appear on a formal tax invoice.

Part 5: The Relational Database Schema (Supabase)

1. Database Philosophy: Immutability & The "Messy Reality"

Financial databases must never delete data. If an invoice is wrong, we don't "delete" it (which breaks the GST sequence); we mark it as `cancelled` or issue a `rectification`. If a client returns goods, we don't just magically erase the ledger balance; we log a `refund` or `credit_note`. This schema is designed to make Chartered Accountants (CAs) love our software because it maintains a perfect, immutable audit trail.

2. Core Tables & Columns

Table 1: `users` (The Master Identity)

- *Purpose:* Firebase handles auth; this table stores the application state.
- **Columns:**
 - `id` (UUID, Primary Key)
 - `firebase_uid` (String, Unique)
 - `email` (String)
 - `phone_number` (String, Unique)
 - `subscription_plan` (Enum: 'free', 'premium')
 - `vapi_wallet_balance` (Decimal, Default: 0.00)
 - `created_at` (Timestamp)

Table 2: `businesses` (The Tally-Compliant Sequence Engine)

- *Purpose:* Stores formal business entities and manages the strict GST invoice numbering.
- **Columns:**
 - `id` (UUID, Primary Key)
 - `user_id` (UUID, Foreign Key)
 - `business_name` (String)
 - `gsttin` (String, Nullable)
 - `logo_url` (String, Nullable)
 - `invoice_prefix` (String, Nullable) - e.g., "RPE/"

- `financial_year_suffix` (String, Nullable) - e.g., "/26-27"
- `next_invoice_sequence` (Integer, Default: 1)
- `created_at` (Timestamp)

Table 3: `contacts` (The Global Address Book)

- *Purpose:* Stores client details.
- **Columns:**
 - `id` (UUID, Primary Key)
 - `user_id` (UUID, Foreign Key)
 - `name` (String)
 - `phone_number` (String)
 - `client_gstin` (String, Nullable)
 - `billing_address` (Text, Nullable)
 - `created_at` (Timestamp)

Table 4: `ledgers` (The Immutable Financial Core)

- *Purpose:* The master record for an invoice or personal lending entry.
- **Columns:**
 - `id` (UUID, Primary Key)
 - `user_id` (UUID, Foreign Key)
 - `contact_id` (UUID, Foreign Key)
 - `business_id` (UUID, Foreign Key, Nullable)
 - `invoice_number` (String, Nullable) - *The sequential ID.*
 - `source_type` (Enum: 'tally_import', 'system_generated', 'manual_entry')
 - `total_amount` (Decimal)
 - `balance_due` (Decimal) - *Current outstanding amount.*
 - `due_date` (Date)
 - `status` (Enum: 'draft', 'pending', 'partially_paid', 'paid', 'overdue', 'cancelled', 'refunded')
 - --- *The CTO "Smart" Additions ---*
 - `is_custom_pdf` (Boolean, Default: false) - *If TRUE, the system bypasses auto-generation. The user has uploaded their own proprietary design.*
 - `pdf_url` (String, Nullable) - *Links to Firebase Storage (either our generated one or the user's custom upload).*
 - `current_version` (Integer, Default: 1) - *Increments if the user rectifies the invoice.*
 - `created_at` (Timestamp)
 - `updated_at` (Timestamp)

Table 5: `ledger_revisions` (The Rectification Engine)

- *Purpose:* If an accountant makes a mistake (wrong amount, wrong GST), they can "rectify" the ledger. We save a snapshot of the *old* version here before updating Table 4. This protects against fraud and keeps CAs happy.
- **Columns:**
 - `id` (UUID, Primary Key)
 - `ledger_id` (UUID, Foreign Key)
 - `previous_total_amount` (Decimal)
 - `previous_pdf_url` (String)
 - `version_number` (Integer)
 - `rectification_reason` (Text) - e.g., "Corrected client GSTIN".
 - `created_at` (Timestamp)

Table 6: `transactions` (Payments, Refunds & Scraps)

- *Purpose:* Replaces the basic "payments" table. This handles money coming IN and money going OUT (refunds).
- **Columns:**
 - `id` (UUID, Primary Key)
 - `ledger_id` (UUID, Foreign Key)
 - `transaction_type` (Enum: 'payment_received', 'refund_issued', 'credit_note_applied', 'bad_debt_writeoff') - *This allows us to handle scrapped invoices or returned orders cleanly.*
 - `amount` (Decimal) - *Can be negative for refunds.*
 - `payment_method` (Enum: 'upi_link', 'cash_manual', 'bank_transfer', 'cheque', 'system_adjustment')
 - `reference_id` (String, Nullable)
 - `logged_at` (Timestamp)

Table 7: `communication_logs` (The Proof of Follow-up)

- *Purpose:* Logs every WhatsApp and AI call.
- **Columns:**
 - `id` (UUID, Primary Key)
 - `ledger_id` (UUID, Foreign Key)
 - `type` (Enum: 'whatsapp_reminder', 'vapi_call', 'email_invoice')
 - `status` (Enum: 'sent', 'delivered', 'read', 'failed', 'call_completed')
 - `cost_deducted` (Decimal)
 - `executed_at` (Timestamp)

3. Core Database Functions (Triggers)

To ensure data integrity, we will write a Supabase Trigger:

- `calculate_balance_due()`: Every time a row is inserted into the `transactions` table (whether it's a payment or a refund), this trigger automatically recalculates the

`balance_due` on the main `ledgers` table and updates its `status` (e.g., if balance hits 0, status becomes 'paid').

4. Row Level Security (RLS)

Every table enforces `user_id = auth.uid()`. Even if a hacker guesses a UUID, the database will return 0 rows unless the request comes from the authenticated user who owns that business profile.

Part 6: The Ingestion Engine (How Data Enters the System)

1. The CTO's Ingestion Philosophy

Frictionless data entry is our biggest competitive advantage. Whether an accountant is uploading 5,000 legacy Tally invoices, or a shop owner is quickly logging a ₹500 udhaari (credit) while a customer is standing at the counter, the system must adapt to *their* speed, not force them to adapt to ours.

2. Flow A: The "Smart" CSV Bulk Importer (For Tally/Busy/Marg Users)

We will not force users to download a rigid template. We will build a 3-step smart importer.

- **Step 1: The Raw Drop:** The user drags and drops their raw `.csv` or `.xlsx` file exported directly from their accounting software.
- **Step 2: AI Column Auto-Mapping (The Magic):** We use a lightweight heuristic script in the frontend. If their CSV has a column named "Party Name", "Client", or "Cust_Name", our system automatically maps it to our `contacts.name` database field. It auto-maps "Bal", "Pending", or "Due Amount" to `balance_due`.
- **Step 3: The Preview & Error Catching:** Before committing to the database, we show a spreadsheet-like preview UI.
 - *The "Partial Success" Rule:* If 490 rows are perfect but 10 have missing phone numbers, the system does **not** crash or reject the whole file. It imports the 490, and highlights the 10 failed rows in red for the user to manually correct right there on the screen.
- **Idempotency (Duplicate Prevention):** If an accountant accidentally uploads the same CSV twice, our backend checks the `invoice_number` and `business_id`. Instead of creating duplicates, it will update the `balance_due` of the existing records.

3. Flow B: The "15-Second" Manual Generator (The Action Button)

For smaller users or personal lending, they need to create a ledger entry instantly.

- **The Global "New Entry" Modal:** Available anywhere on the dashboard.
- **Inline Contact Creation:** The user types a phone number. If it exists in their `contacts` table, it auto-fills the name. If it doesn't exist, they just type the name, and the system seamlessly creates a new contact *and* the ledger entry in one single backend transaction.
- **The "Attach PDF" Toggle:** At the bottom of this form, there is a toggle: `[Generate Tax Invoice] | [Upload Custom PDF] | [No PDF, Just Amount]`. This supports the dynamic rules we defined in Part 4 and Part 5.

4. Flow C: The "Lazy" WhatsApp Ingestion (The CTO's Secret Weapon)

Business owners are busy. Sometimes they don't even want to open the web dashboard.

- **How it works:** A user can simply forward an existing PDF bill (generated from any software) directly to the **Recoverpe Official WhatsApp Bot** from their registered mobile number.
- **The Workflow:**
 1. They forward the PDF.
 2. They type: `"70211XXXXX 45000"` (Phone number and Amount).
 3. Our backend receives this via Meta Webhook, uploads the PDF to Firebase Storage, automatically creates the `ledger` entry, links the PDF, and immediately sends the payment link to that client's phone number. *This is pure magic for the user and requires zero frontend interface.*

5. Technical Safeguards (The Queue System)

- **Protecting Meta API Limits:** If a user uploads a CSV with 2,000 pending invoices, triggering 2,000 WhatsApp messages in one second will get our WhatsApp Business account banned for spam.
- **The Background Worker:** We will implement a queue system (using a service like Upstash or Supabase Edge Functions). When a massive CSV is uploaded, the database commits instantly, but the WhatsApp messages are queued and released at a rate of 10-15 messages per second.

Part 7: Zero-KYC Payment Routing (The Instant Settlement Engine)

1. The CTO's Payment Philosophy

The biggest friction for an MSME signing up for software is the payment gateway KYC (which can take 7 days and fail arbitrarily). We are eliminating this. During onboarding, the business owner simply inputs their existing VPA (Virtual Payment Address / UPI ID) like

rahulenterprises@sbi. That's it. No documents, no approvals. They can start collecting money in 10 seconds. We are a software provider, not a payment aggregator.

2. The UPI Intent Engine (How it works under the hood)

When an invoice is generated, our backend dynamically creates a strict UPI Intent URI.

- **The Structure:**
`upi://pay?pa={merchant_vpa}&pn={merchant_name}&am={balance_due}&tr={invoice_id}&tn=Payment_for_Recoverpe`
- **Mobile Experience (90% of cases):** When the debtor clicks "Pay Now" on WhatsApp, this link triggers their phone's OS to pop up an action sheet offering Google Pay, PhonePe, Paytm, or CRED. The amount and merchant name are locked and pre-filled.
- **The Desktop Fallback (The 10% Edge Case):** If a corporate client clicks the link on WhatsApp Web (Desktop), the `upi://pay` link will break because desktops don't have UPI apps.
 - *The CTO Solution:* The link first routes to a lightweight Recoverpe landing page (`recoverpe.com/pay/{invoice_id}`). Our frontend detects the device. If it's a mobile device, it auto-redirects to the UPI app. If it's a desktop, it dynamically renders a **Scannable QR Code** on the screen so the corporate client can scan it with their phone.

3. Dynamic QRs on PDFs

The same URI string generated above is passed through a lightweight QR Code generator library in our backend. This QR code is stamped directly onto the bottom right corner of the Tax Invoice PDF. Even if the user prints the PDF and hands it to a customer physically, the customer can scan the paper to pay exactly the right amount.

4. The Partial Payment Dilemma

Sometimes a client owes ₹50,000 but only has ₹20,000 today.

- Our payment landing page (`recoverpe.com/pay/{invoice_id}`) features an "Edit Amount" toggle.
- If the client changes the amount to ₹20,000, our system generates a *new* UPI Intent link on the fly with `am=20000.00`. This ensures the transaction doesn't fail due to amount mismatches in the banking system.

5. The "Missing Webhook" Reconciliation Problem (How we know they paid)

Because the money travels straight from HDFC to SBI, Dahikar Technologies does not get an API notification. How do we update the ledger from "Pending" to "Paid"?

- **The "Proof of Payment" WhatsApp Loop:**

1. 15 minutes after the debtor clicks the "Pay Now" link, our Meta API sends an automated follow-up: *"If you have completed the payment, please reply with 'Done' or upload a screenshot of the transaction."*
2. If the client uploads a screenshot, our backend receives the image via Meta Webhook.
3. The image instantly appears on the MSME Owner's Dashboard with a bright yellow "Verification Pending" badge.
4. The owner looks at the screenshot, checks their bank app, and clicks **"Approve"**.
5. *Backend Action:* This single click creates a record in the `transactions` table (Part 5) and drops the ledger `balance_due` to 0.

6. Future Expansion (V2 - Automated Bank Sync)

In the future, we can upsell a premium feature: Integration with Account Aggregators (like Setu or Finvu). This would allow the business owner to securely link their read-only bank statement to Recoverpe, allowing our AI to automatically spot the exact amount and UTR number, making reconciliation 100% automated without screenshots. For V1, the 1-click manual approval is perfect.

Part 8: The Omnichannel Escalation & Communication Engine

1. The "Sender" Philosophy & The Omnichannel Fallback

Because Meta's Cloud API requires us to send automated messages from a central, verified "Recoverpe" business number, we must bridge the gap between automation and personal touch, while ensuring 100% deliverability even if the debtor isn't on WhatsApp.

- **Channel 1: WhatsApp (Primary 90%):** Sent via the Recoverpe Official Bot. *Format:* "Message from [User's Business Name] via Recoverpe."
- **Channel 2: Fast2SMS Text Fallback (The 5%):** If our WhatsApp webhook detects a "Number not on WhatsApp" error, the system instantly falls back to a standard DLT-approved SMS containing the payment link.
- **Channel 3: Email Delivery (The B2B 5%):** For corporate clients, the system simultaneously emails the PDF invoice and payment link via AWS SES / Resend.
- **The "Personal Number" Workaround:** On the user's dashboard, next to every overdue invoice, is a **"Send from my Phone"** button. Clicking this does not use our API. Instead, it generates a pre-filled `wa.me` deep link that instantly opens the user's *own* WhatsApp app on their phone with a perfectly drafted message and UPI link, allowing them to send it personally.

2. Party-Specific Cadence (The "Aggression" Dial)

Not all debtors are the same. A 10-year loyal customer needs different treatment than a first-time buyer who is ignoring calls. Instead of a global timeline, the user can set a **Cadence Rule** for specific contacts or specific invoices:

- **VIP Mode:** Send the invoice once. Never send automated reminders. (User will handle it manually).
- **Standard Mode:** Remind 3 days before, on the due date, and once a week after.
- **Aggressive Mode ("Vasuli Mode"):** For high-risk defaulters. Send a reminder *every alternate day*, escalating to *every single day* once it crosses 30 days overdue.

3. The Multi-Stage Escalation Templates

Templates must adapt to the age of the debt. A 90-day overdue message must sound legally intimidating, not friendly. Our Next.js backend will dynamically select the template based on the `days_overdue` integer.

- **Stage 1: The Professional Issuance (Day 0)**
 - *Body:* "Hello {{1}}, an invoice of ₹{{2}} has been generated by {{3}}. The due date is {{4}}. You can download your detailed bill and pay instantly using the secure link below."
 - *Buttons:* [Pay Now] | [View PDF]
- **Stage 2: The Friendly Nudge (Day 1 to Day 15 Overdue)**
 - *Body:* "Hi {{1}}, this is a reminder from {{2}} that your ledger shows a pending balance of ₹{{3}}, which was due on {{4}}. Please let us know if you face any issues, or click below to settle via UPI."
 - *Buttons:* [Pay Now] | [Will pay in 2 days] (Quick Reply)
- **Stage 3: The Firm Warning (Day 30 to Day 60 Overdue)**
 - *Body:* "Dear {{1}}, your payment of ₹{{2}} to {{3}} is now severely overdue. Continuous non-payment impacts our ability to supply further goods/services. Please clear this immediately to keep your ledger in good standing."
 - *Buttons:* [Pay Immediately]
- **Stage 4: The High-Risk Escalation (90+ Days Overdue)**
 - *Body:* "URGENT: {{1}}, your outstanding balance of ₹{{2}} with {{3}} is over 90 days past due. Please settle this by end of day to avoid formal recovery escalation and potential impact on your business credit record."
 - *Buttons:* [Resolve Dues Now]

4. The Two-Way Webhook Engine (Listening & Reacting)

Sending is only half the battle. Our `/api/webhooks/meta` endpoint makes the system intelligent:

- **Read Receipts:** We capture the blue ticks. The user's dashboard will show: "*Rahul read your final warning yesterday at 4:30 PM.*" This removes the debtor's ability to say "I didn't see the message."
- **The Screenshot Loop:** If the debtor replies with an image (payment screenshot), our backend hits the Graph API, downloads the encrypted image, pushes it to

Firebase Storage, and flags the invoice on the user's dashboard as "Verification Pending" (as defined in Part 7).

- **The "Opt-Out" Shield:** If a debtor types "STOP" or "BLOCK", our system auto-flags their number in the `contacts` table. We immediately cease all automated WhatsApps and seamlessly switch to SMS/Email only, protecting our Meta account health score from being banned.

5. Smart Throttle & Rate Limits

If an MSME sets 500 invoices to "Aggressive Mode", they will all trigger at 9:00 AM.

- **The Upstash Redis Queue:** We will push all outgoing messages into a Redis queue. The queue trickles them out across WhatsApp, SMS, and Email at a safe rate of 10 messages per second, ensuring we never hit API rate limits or trigger spam algorithms.

Part 9: The VAPI AI Voice Caller & The Profit Engine

1. The CTO's Economics: The "Recoverpe Credit" System (40% Margin)

We will not charge users directly in Rupees-per-minute for AI calls. This invites unwanted cost comparisons. Instead, we are implementing a **Prepaid Credit Wallet**.

- **The Math:** 1 Recoverpe Credit = 1 Minute of AI Talk Time.
- **The Pricing Tier:** Users buy packages via Razorpay (e.g., 100 Credits for ₹1,000). So, we charge them ₹10 per minute.
- **The Backend Reality:** Our actual cost (Vapi API + Indian Exotel/Twilio Telephony routing) is approximately ₹6 per minute.
- **The Profit:** Dahikar Technologies locks in a guaranteed ₹4 profit on every single minute of AI calling (a clear 40% gross margin) *before* the call even happens.

2. Mindblowing Feature 1: The "Live Warm Transfer" (The Bridge)

MSMEs hate AI that hits a dead end. If the debtor says, *"I won't pay because the last shipment had 10 broken items,"* an average bot just says, *"I will note that down."* Our AI is smarter.

- **The Logic:** We inject a specific function-calling rule into the VAPI prompt.
- **The Action:** If the AI detects a "Dispute" or the debtor demands to speak to the boss, the AI responds: *"I understand there is a discrepancy. Please hold the line, I am bridging this call to the accounts manager directly."*
- **The Magic:** VAPI instantly dials the Business Owner's registered mobile number, whispers *"Transferring a disputed invoice call from [Debtor Name]"*, and connects them. The bot acts as a true digital receptionist.

3. Mindblowing Feature 2: AI Sentiment & Flight Risk Profiling

VAPI does not just transcribe text; it understands the acoustic properties of the call (interruptions, volume, tone).

- **Post-Call Analysis:** In the webhook payload, we will ask the LLM to rate the debtor's sentiment.
- **Dashboard Badges:** Based on the call, the ledger on the user's dashboard gets an auto-generated tag:
 -  *Cooperative* (Debtor was polite, promised a date).
 -  *Evasive* (Debtor gave vague answers, tried to change the subject).
 -  *Flight Risk / Hostile* (Debtor yelled, hung up, or outright refused).
- This gives the business owner unprecedented psychological leverage before they decide to take legal action.

4. Triggering the AI (The Escalation Protocol)

The AI is a premium escalation tool. It is triggered by the "Aggressive Mode" Cadence rules defined in Part 8.

- **Dynamic Prompt Injection:** When the Cron Job fires, our Next.js backend sends a POST request to VAPI with hyper-specific contextual data.
- **The Master Prompt:** *"You are 'Sneha', a firm accounts recovery agent for {{business_name}}. You are calling {{debtor_name}}. They owe ₹{{balance_due}} which is now {{days_overdue}} days late. Your goal is to extract a strict 'Promise to Pay' date. Do not offer discounts. If they claim a dispute, use the 'transfer_to_owner' function. Speak in conversational Hindi and English."*
- **Multi-Lingual Fallback:** The AI starts in English/Hindi. If the debtor replies in Marathi or Gujarati, the LLM is instructed to seamlessly switch its language to match the debtor, building instant rapport (or pressure).

5. The VAPI Webhook (Closing the Loop & Billing)

When the debtor hangs up, Vapi sends the webhook back to </api/webhooks/vapi>.

- **Step 1: Profit Deduction:** We check the precise call duration (e.g., 2.5 minutes). We deduct 3 Credits (rounding up to the nearest minute, maximizing our margin) from the user's [vapi_wallet_balance](#).
- **Step 2: Dashboard Logging:** We insert the record into [communication_logs](#).
- **Step 3: The "Executive Summary" UI:** On the dashboard, the user sees an audio player of the recording, the Sentiment Badge (e.g.,  *Hostile*), and a 2-sentence AI summary: *"Sneha connected with Rahul. Rahul became aggressive and claimed he already paid in cash last week. Call was flagged for manual review."*

6. Legal & Compliance Constraints (TRAI)

- Because this is automated outbound calling in India, we must ensure our telephony provider uses **Transactional DLT Routing** (not promotional), ensuring we bypass DND (Do Not Disturb) registries legally, as debt recovery is a transactional update.
- **Curfew Rules:** Our backend strictly suppresses any API call requests between 7:00 PM and 9:00 AM IST to comply with RBI/TRAI guidelines on harassment.

Part 10: The Automation Engine & Cron Scheduler (The Heartbeat)

1. The Serverless Challenge & The "pg_cron" Solution

Next.js (our frontend/backend) is serverless, meaning it cannot "wake itself up" at 8:00 AM every day to run a loop.

- **The 10/10 CTO Solution:** We are using **Supabase pg_cron**. Because our database is PostgreSQL, we will write a database-level cron job. Every day at precisely 8:30 AM IST, the database wakes up, looks at the date, and triggers a secure HTTP POST request to our Next.js backend (`/api/cron/daily-runner`).

2. The Master Query (Finding the Targets)

When the Next.js API is triggered, it does not just blindly select all invoices. It runs a highly optimized SQL query against the `ledgers` table combined with the `businesses` (Cadence Settings) table.

- **The Conditions:** It looks for ledgers where:
 - `status` = 'pending' OR 'overdue' OR 'partially_paid'.
 - AND the current `date` matches the user's specific trigger rules (e.g., Today is exactly 3 days before `due_date`, OR Today is exactly 30 days after `due_date`).
 - AND `communication_paused` = FALSE (User hasn't manually muted this client).

3. The Queue & Throttling System (Protecting the Meta Account)

If the query finds 10,000 invoices that need reminders today, we *cannot* fire 10,000 Meta API requests at 8:30:01 AM. Meta will flag us as a DDoS attack and ban our WhatsApp number.

- **Upstash QStash (The Line Manager):** The Master Query takes those 10,000 targets and pushes them into an Upstash Redis Queue.

- **The Drip Feed:** QStash sends them to our Meta sender API at a safe, controlled rate of **15 messages per second**. This guarantees 100% deliverability without triggering spam alarms.

4. Strict Idempotency (The "Double-Send" Shield)

The worst bug a recovery software can have is sending the same "Final Warning" 5 times in a row. It ruins the user's relationship with their client.

- **The Lock Mechanism:** Before sending *any* message, the worker checks the `communication_logs` table.
- **Rule:** If an entry exists for `ledger_id = 123` and `action = 'Day_15_Reminder'` with a timestamp of today, the system gracefully skips it. Even if the Cron Job accidentally fires twice, the client is completely protected.

5. Timezone Enforcement (IST Lock)

Because servers run in UTC, dates can get messy. (8:00 AM IST is 2:30 AM UTC).

- All backend date calculations will use the `date-fns-tz` library strictly locked to `Asia/Kolkata`.
- **The Curfew:** A hardcoded middleware rule ensures that *zero* automated outgoing communications (WhatsApp, SMS, or VAPI Calls) can be executed between **8:00 PM and 8:00 AM IST**, ensuring strict compliance with TRAI harassment laws.

6. The Mindblowing Feature: AI Cashflow Predictor (The Engagement Hook)

During the daily 8:30 AM Cron run, a secondary asynchronous job scans the entire `transactions` history for Premium Users.

- **The Insight Engine:** It calculates the *average payment delay* for specific contacts.
- **The Dashboard Alert:** When the user logs in, instead of just seeing "₹5 Lakhs Due", the AI pops up a notification:
 - *Insight:* "You have ₹50,000 due from Rohit Traders today. Historically, Rohit pays an average of 14 days late. Projected arrival date is the 26th."
- This feature pivots the software from a "dumb reminder tool" to an "Intelligent CFO," keeping users deeply engaged without diluting our core product into an expense tracker.

Part 11: The Edge Case Manager (Handling the Messy Reality)

1. The CTO's Reality Check

Automation is only as good as its manual override. The system must instantly react to off-platform events (cash payments, phone call negotiations) to prevent the AI or WhatsApp bot from sending embarrassing or harassing messages to clients who have already settled.

2. The "Offline Settlement" Override (The Kill Switch)

When a user receives ₹50,000 in physical cash or via a direct NEFT that bypassed our UPI link, they must be able to stop the automation instantly.

- **The UI Action:** A prominent **"Log Offline Payment"** button next to every pending invoice on the dashboard.
- **The Data Capture:** The user enters the Amount (e.g., ₹50,000), Method (Cash, Cheque, Bank Transfer), and an optional Reference Note (e.g., "Cheque #12345").
- **The Backend Reaction:**
 1. The `transactions` table logs the entry.
 2. The `ledgers` table hits `balance_due = 0` and `status = paid`.
 3. *The Kill Switch:* The system instantly deletes any pending QStash queue jobs for this ledger. Tomorrow's 8:30 AM Cron Job will completely ignore this invoice.
 4. *The Courtesy Message:* The system fires one final WhatsApp: *"Payment of ₹50,000 received via [Method]. Your ledger is now clear. Thank you for your business!"*

3. The "Interactive Extension" Loop (The 10/10 Negotiation Engine)

Indian MSMEs operate on relationships and credit extensions ("Bhai, 4 din de do"). We are turning our WhatsApp bot into a negotiation mediator.

- **The Debtor Experience:** When the Day 0 (Due Date) WhatsApp reminder is sent, it includes two Interactive Meta Buttons:
 - [Pay Now]
 - [Request Extension]
- **The Interactive Flow:** If the debtor clicks [Request Extension], the bot auto-replies: *"How many days of extension do you need? (Reply with a number, e.g., 5)"*.
- **The Dashboard Approval:** The business owner gets a push notification / dashboard alert: *"Rohit Traders is requesting a 5-day extension for Invoice #102."*
 - If the owner clicks [Approve]: The system updates the `due_date` in the database to +5 days. All automated reminders and AI calls are instantly rescheduled. The debtor gets a WhatsApp: *"Your extension has been approved. New due date is [Date]."*
 - If the owner clicks [Reject]: The debtor gets: *"Extension denied. Please clear the dues today to avoid escalation."*

4. Dynamic Partial Payments (The "Running Balance" Fix)

If a client owes ₹1,00,000 but transfers ₹40,000 today.

- **The Math Engine:** The owner logs the ₹40,000 partial payment (or approves the WhatsApp screenshot).
- **Status Shift:** The ledger `status` changes from `pending` to `partially_paid`.
- **Dynamic Template Injection:** The Cron Job knows *not* to ask for ₹1,00,000 anymore. The next scheduled reminder automatically adjusts its variables:
"Reminder: We received your partial payment, but a balance of ₹60,000 is still pending for Invoice #102..."

5. The "Dispute & Mute" Protocol (The VIP Shield)

Sometimes a high-value client gets angry because a shipment was delayed, and the business owner needs to handle them delicately on a personal call.

- **The Mute Toggle:** On the dashboard, every contact and ledger has a `Pause Automations` toggle switch (changing the database flag `communication_paused = TRUE`).
- **The Effect:** This acts as a complete shield. The invoice is still marked as "Overdue" internally so the owner doesn't lose track of the money, but the Cron Job and VAPI AI are strictly forbidden from contacting this person until the owner toggles it back off.

6. The "Bounce" Fallback (Bad Data Handling)

What if the CSV upload contained a fake number, or a landline number?

- **The Meta Webhook Catch:** When Meta attempts to send the message and fails (Error: "User not on WhatsApp"), our webhook catches the error instantly.
- **The Auto-Pivot:** The system flags the `phone_number` as invalid for WhatsApp. It immediately routes the message payload to **Fast2SMS (DLT Text Message)** as a fallback, and puts a small red warning icon next to the client's name on the dashboard: *"WhatsApp delivery failed. Routed via SMS."*

Part 12: The User Dashboard & Analytics (The Dopamine Engine)

1. The CTO's Dashboard Philosophy: Action > Information

MSME owners do not have time to read complex pie charts. They want to open the web app on their phone and instantly know three things: *Who owes me money? Who paid me today? What needs my attention right now?* The entire UI will adhere strictly to our Black, White, and Grey design system, using the `state-success` (Green) color *only* to highlight money we successfully recovered.

2. The Global Workspace Toggle (The Context Switcher)

At the very top center of the dashboard sits the toggle defined in Part 4: [**Personal**] | [**Business**].

- **The Magic:** Clicking this toggle doesn't reload the page. It instantly re-fetches the Supabase data, updating every number, chart, and list on the screen to reflect either their personal lending or their formal business ledgers.

3. The "Hero" Metrics (Proving our ROI)

At the top of the dashboard are three massive, bold metric cards.

- **Card 1: Total Outstanding:** The sum of all **balance_due** across pending ledgers. (Color: Brand Black).
- **Card 2: Severely Overdue:** The sum of money that has crossed the due date. (Color: Brand Black with a subtle red warning icon).
- **Card 3: Recovered via Recoverpe (The Dopamine Hit):** The total amount of money settled *since* they installed our software. (Color: Solid Black card with bright White and Green text). *Psychological effect: Every time they see this number go up, our ₹1,999/month subscription feels like a bargain.*

4. The "Triage Zone" (Urgent Action Required)

Below the Hero Metrics, we do not just show a list of invoices. We show an intelligent "Triage" list based on the Edge Cases from Part 11.

- **Pending Verifications:** "Rahul uploaded a screenshot for ₹10,000. [Verify & Clear]"
- **Extension Requests:** "Surat Textiles is asking for 3 more days. [Approve] | [Reject]"
- **VAPI Escalation Required:** "Sneha (AI) flagged a call with Amit as 'Hostile'. [Listen to Audio] | [Send Final Legal Warning]"

5. The "Wall of Shame" (Top Defaulters Widget)

A horizontally scrollable list (mobile) or grid (desktop) highlighting the top 5 clients who owe the most money for the longest time.

- **The 1-Click Escalation:** Next to each name is a bold black button: [**Escalate**]. Clicking this opens a modal allowing the owner to instantly override the standard cadence and trigger a VAPI AI phone call or a fast-tracked SMS warning.

6. Technical Implementation: Supabase vs. Firebase

It is critical that Cursor understands the separation of concerns for analytics:

- **Financial Analytics (Supabase RPCs):** We will *not* pull 5,000 ledgers to the Next.js frontend to calculate the "Total Outstanding". That crashes the browser. We will write Supabase **RPCs (Remote Procedure Calls)**. The database runs the heavy SQL

aggregation internally and sends just the final number (e.g., ₹5,40,000) to the frontend in milliseconds.

- **Behavioral Analytics (Firebase):** We use Firebase Analytics under the hood to track *how* the user uses the app. We log custom events: `button_click_escalate`, `invoice_generated_manual`, `csv_uploaded`. This gives us (the admins) the data to know which features are actually being used so we can improve them.

7. The AI Cashflow Health Score

Injecting the insight engine from Part 10 directly into the UI.

- The dashboard features a circular progress bar (e.g., "72/100 Cashflow Score").
- Below it, the AI generates one dynamic sentence daily: *"Your recovery rate is improving, but 40% of your capital is stuck with clients over 60 days late. Consider pushing the 'Wall of Shame' to Aggressive Mode."*

Part 13: File Storage & PDF Generation (The Document Engine)

1. The CTO's Technical Pivot: Escaping the Serverless Trap

We will absolutely *not* use headless browsers (Puppeteer/Playwright) to generate PDFs. They exceed Next.js serverless size limits.

- **The 10/10 Solution:** We will use `@react-pdf/renderer`. This library allows us to write the invoice layout using standard React components (`<View>`, `<Text>`, `<Image>`). It compiles directly to a raw PDF byte stream in milliseconds, either on the server or instantly on the client's device, costing us zero extra computing power.

2. The Firebase Storage Bucket Architecture

If we just dump everything into one folder, our Firebase storage costs will explode, and data will leak. We will structure the Firebase Cloud Storage into strict, purpose-built buckets with specific lifecycle rules.

- `/public/logos/{business_id}.png`: Stores the MSME's company logo. Publicly readable so it loads fast on the invoice.
- `/secure/invoices/{ledger_id}.pdf`: System-generated or user-uploaded PDFs. Strictly protected by Firebase Security Rules. Only the business owner and the recipient with the exact signed URL can view it.
- `/secure/proof_of_payments/{ledger_id}/{timestamp}.jpg`: The screenshots debtors send via WhatsApp.

- `/temp/csv_imports/{user_id}/{timestamp}.csv`: The messy Tally exports users upload. *CTO Rule*: We will attach a Firebase Cloud Function Lifecycle Rule to this folder: **Auto-delete after 24 hours**. Once the data is ingested into Supabase (Part 6), the raw CSV is useless garbage. Deleting it saves us massive storage costs.

3. The Dynamic GST Engine (Smart Rendering)

The PDF isn't just a static template; it's a dynamic tax calculator. When the user hits "Generate", the engine checks the Supabase `businesses` table and the `contacts` table.

- **The State Code Check**: It compares the first 2 digits of the user's GSTIN (e.g., `27` for Maharashtra) with the buyer's GSTIN.
- **Intra-state (Same State)**: The PDF dynamically splits the tax into two columns: **CGST (9%) and SGST (9%)**.
- **Inter-state (Different State)**: The PDF dynamically merges it into one column: **IGST (18%)**.
- This feature alone will make Indian Chartered Accountants recommend our software to their clients, as it prevents major compliance errors.

4. The QR Code Injection Protocol

As defined in Part 7, we generate a Zero-KYC UPI Intent Link.

- Before the PDF is rendered, we pass that `upi://pay` string through a lightweight `qrcode` library.
- The resulting Base64 image is stamped directly onto the bottom right of the PDF under the heading: **"Scan to Pay Exact Amount."**

5. The "Bring Your Own PDF" (BYOP) Pipeline

10% of high-end corporate users will say, *"I cannot use your PDF template, my enterprise clients require my specific format with my digital signature."*

- **The Override**: In the manual creation modal, we provide a "Upload Custom Bill" drag-and-drop zone.
- **The Flow**: The user uploads their `.pdf`. We push it straight to `/secure/invoices/`. We generate the Supabase `ledger` entry so we can track the money, but we *bypass* the `@react-pdf/renderer` entirely. When the WhatsApp message goes out, it links to *their* PDF, but still uses *our* automated reminders and UPI links.

6. The "Invisible Watermark" (The Viral Growth Loop)

For users on the "Free/Basic" tier of our software, every generated PDF will have a very subtle, clean text at the absolute bottom footer:

- *"Powered by Recoverpe.com - Smart Ledgers & Automated Collections"*
- This turns every single invoice our users send out into a billboard for our software. If their clients like the smooth UPI payment experience, they will look at the footer and sign up for their own business.

Part 14: Monetization Strategy & Razorpay Integration (The Profit Engine)

1. The CTO's Indian SaaS Philosophy: Value Before Extraction & Ethical Upsells

Indian MSMEs do not pay for "features"; they pay for ROI. Our monetization architecture is built on a "Freemium to Premium" pipeline, but with a crucial psychological twist: **We must show them the money we recovered for them *before* asking for a subscription.**

Furthermore, any upsells we offer must provide 10x value through automation, without "looting" the user with artificial agency fees. All payments are handled securely via Razorpay.

2. Tier 1: "The Seed" (Free Plan - The Viral Loop)

The goal of this tier is to acquire users for ₹0 Customer Acquisition Cost (CAC) and prove our software works.

- **What they get:**
 - Up to 15 Active Business Invoices at any time.
 - Standard WhatsApp reminders (Days 0, 3, 7 only).
 - **The Dopamine Hit:** They *do* get full access to the "Money Recovered" dashboard widget. They need to see that big green number ticking up to realize our worth.
- **The Restriction (The Hook):**
 - NO AI Voice Calls.
 - NO "Aggressive Mode" daily reminders.
 - Limit of 15 invoices (When they try to add the 16th, the paywall hits).
- **The Viral Tax:** Every PDF invoice and every WhatsApp message sent on the free tier contains our permanent watermark: *"Recovered instantly via Recoverpe.com. Click to automate your collections."*

3. Tier 2: "The Engine" (Premium Plan - ₹1,999/month)

This is the core recurring revenue engine, processed via **Razorpay Subscriptions** (UPI Autopay or Credit Card mandates).

- **The Psychology:** If an MSME has ₹5 Lakhs stuck in the market, ₹1,999/month to automate the recovery is an absolute no-brainer.
- **What they get (Unlimited Power):**
 - Unlimited Invoices and CSV Ingestion.
 - Customizable WhatsApp Cadence ("Aggressive Mode").
 - Complete removal of Recoverpe branding.
 - Predictive AI Analytics (The Cashflow Score).
 - Access to the VAPI AI Wallet.

4. Dynamic Discounting & The "Proof of Value" Trigger

We don't do standard 7-day trials. We do "Success-Based" trials.

- **The Trigger:** The moment a Free user successfully recovers their first ₹50,000 using our app, the dashboard triggers a celebratory confetti animation.
- **The Modal:** *"Recoverpe just helped you collect ₹50,000! We take 0% commission. Upgrade to Premium in the next 24 hours to unlock unlimited invoices and claim 50% off your first month."*
- **The Annual Anchor:** We display the monthly plan at ₹1,999/mo. Right next to it, we display the Annual Plan at **₹17,999/year (Effectively ₹1,499/month)**. This "anchor" pricing pushes users to pay Dahikar Technologies upfront cash for the whole year.

5. The Consumable: VAPI AI Prepaid Wallet (The 40% Margin)

As defined in Part 9, AI minutes are a premium consumable.

- **The Interface:** A "Wallet" tab showing "Recoverpe Credits."
- **Razorpay One-Time Checkout:** Users buy credit packs:
 - ₹1,000 for 100 Credits (100 Minutes).
- **The Profit Math:** Our actual telecom + API cost is ₹6/minute. By charging ₹10/minute via Credits, Dahikar Technologies locks in **₹400 of pure profit** every time a user clicks "Recharge ₹1,000," *before a single call is made*.

6. THE WOW FACTOR 1: The "1-Click Legal Notice" (85% Profit Margin)

- **The Feature:** A massive red button on 90+ day overdue invoices: [**Issue Legal Notice**].
- **The Micro-Transaction:** Triggers a Razorpay popup for a one-time fee of **₹999**.
- **The Execution:** Our backend automatically generates a highly formal "Final Demand Notice" PDF on a digital Advocate's letterhead, citing the *Negotiable Instruments Act*. We automatically email and WhatsApp this to the debtor.
- **The Profit:** Server cost: ₹0. We charge ₹999. Pure margin.

7. THE WOW FACTOR 2: The MSME Samadhaan "Smart-Kit" (₹499 - 100% Margin)

We will not overcharge the user or use human agents. We provide a completely automated, ethical, and empowering self-serve digital kit.

- *The Problem:* The Govt Samadhaan portal asks for strict proof (Invoices, delivery challans, communication history) and is confusing to navigate.
- *Our Solution (The ₹499 Micro-Transaction):* When the user hits the [File Samadhaan] button and pays ₹499, our backend instantly does two things:
 1. **The Evidence Docket:** It merges their original PDF invoice, a timeline of our automated reminders, and the critical **WhatsApp Blue-Tick Read Receipts** into a single, perfectly formatted PDF optimized for the government portal's size limits.
 2. **The Action Guide:** We unlock a "Guide Mode" overlay on their screen. It provides the exact URL to the government portal, a short 60-second video on where to click, and a "Copy" button for all their relevant data (GST, Invoice numbers) so they can just paste it into the government site in 3 minutes.
- *The Result:* High user trust, zero human intervention from our team, and 100% scalable profit.

8. Technical Implementation: The Razorpay Webhook Shield

To prevent fraud, the frontend *never* dictates user permissions.

- **The Secure Flow:** When a payment succeeds, Razorpay sends a secure Webhook to our `api/webhooks/razorpay` endpoint.
- Our server verifies the cryptographic signature. Only then does it update the database (`subscription_plan = 'premium'`, or `vapi_wallet_balance += 100`, or triggers the legal PDF generation).

Part 15: The Super Admin Panel (The Zero-Trust Fortress & Operations Hub)

1. The CTO's Security Philosophy: "Trust No One, Verify Everything"

The Admin Panel (`admin.recoverpe.com`) is a completely separate web application. It does not run on the same server as the main user app.

- **The Moat (Hacker-Proofing):** It is locked behind Google Workspace SSO, enforces strict Hardware 2FA, and is **IP-Whitelisted** (can only be accessed from the Dahikar Technologies office router or your personal home IP).
- **The Immutable Audit Trail:** If an employee clicks *anything*—views a user's ledgers, issues a refund, or pauses a bot—it is permanently recorded in the `admin_audit_log` with their name and timestamp.

2. Role-Based Access Control (RBAC)

You can safely hire support staff without handing over the keys to the kingdom.

- **The "Founder" Role (God Level):** You. Can view global revenue, issue server-side discounts, trigger the Meta Kill Switch, and export the database.
- **The "Support Tier 1" Role:** Your employees. They can search for a user by email/phone. They can see if a WhatsApp reminder failed to send, but the actual PDF amounts and contact names are heavily masked (R***** Traders, ₹, ***). They *cannot* delete accounts or issue discounts.
- **The "Financial Auditor" Role:** A read-only dashboard for your CA to view Razorpay settlement data and API burn rates for tax filing.

3. The Global Telemetry Dashboard (The "Pulse")

When you log in, you see the exact heartbeat of your bootstrapped empire.

- **Lifetime Recovery Volume:** "Recoverpe has settled ₹45.2 Crores to date." (Your main pitch deck metric).
- **User Velocity:** Active Free Users vs. Active Premium Users.
- **The Real-Time Margin Tracker:** A live ticker showing:
 - *Money In (Today):* Razorpay Subscriptions + VAPI Wallet Recharges.
 - *Money Out (Today):* VAPI AI minutes burned + Meta API costs.
 - *Net Daily Profit:* ₹18,450.

4. Advanced User Management & The "Profile" Engine

You rightly noticed the main app needs a robust "My Profile" section for the business owner. The Admin Panel serves as the backend controller for this.

- **The Profile Override:** If a user accidentally locks themselves out or types their GSTIN wrong during setup, you (Founder) can enter their profile backend and force-update their verified data.
- **The "Ghost Mode" Impersonation:** Click to securely mirror a user's dashboard as a *read-only* ghost. You see exactly what they see (to debug a visual bug) without asking them for their password.
- **The Ban Hammer:** If a user is uploading fake CSVs or using our API to harass people, click [**Suspend User**]. This instantly kills their login session, pauses all Cron Jobs, and revokes their WhatsApp token.

5. DPDP Compliance & The "Delete Account" Purge Engine

To comply with India's strict Digital Personal Data Protection Act, users will have a "Delete My Account & Data" button in their Profile settings. The Admin Panel handles the fallout safely.

- **The Soft-Delete Window (30 Days):** When a user clicks "Delete", their account is immediately hidden and they are logged out. The Admin Panel flags them in a "Pending Purge" list. They have 30 days to email us to recover it if it was a mistake.
- **The Hard Purge (Automated):** On Day 31, the system automatically scrubs their Firebase Auth, deletes their generated PDFs from the storage bucket, and overwrites

their personal data in the Supabase tables with `[Deleted User]`, ensuring your database remains legally compliant without breaking global financial math.

6. "Leak-Proof" Server-Side Discounts (The Bootstrapped Hustle)

We will never use shareable text coupon codes like "SAVE50".

- **The Feature:** You want to reward a free user who just recovered ₹50,000 to push them to upgrade.
- **The Execution:** In the Admin Panel, you filter for "Free users who recovered >₹50k". You click a button called `[ Grant 24Hr 50% Entitlement ]`.
- **The Magic:** The backend silently flips a boolean (`eligible_for_discount = true`) on their specific User ID. When *they* open their app, a massive banner says "Congratulations on your recovery! Here is a 50% discount locked to your account." The checkout page dynamically halves the price. No code is ever generated, and it is impossible to share it with anyone else.

7. AI Token & Telephony Management (Protecting the Bank)

Vapi and Meta are your biggest variable costs.

- **Global Burn Rate Widget:** Shows total AI minutes consumed across the entire platform today.
- **The "Runaway AI" Circuit Breaker:** If a single user suddenly burns 100 AI minutes in an hour (possible bug or abuse), the Admin Panel flashes red, instantly auto-pauses their AI access, and texts your personal phone to investigate.

8. The Meta Shield & The Global Kill Switch (DEFCON 1)

Our entire business relies on Meta keeping our "Recoverpe Official" WhatsApp number active.

- **The Health Monitor:** The backend tracks the "Bounce Rate" and "Block Rate" of our WhatsApp webhook live.
- **The Global Kill Switch:** A massive red button on the admin dashboard. If Meta threatens a ban due to high spam rates from our users, you click this. It instantly flushes the Upstash Redis Queue and hard-stops *all* outgoing WhatsApp and SMS messages platform-wide until you resolve the issue, saving the core business from permanent deletion.